

Etapa 3 — Implementação do Classificador Naive Bayes em SQL

Autor Marcus Vinícius Santos de Almeida	Data de entrega 04/09/2026
Banco de dados SQLite — <code>dados/classificador.db</code>	Entrada de treino <code>etapa2_dados_treinamento.csv</code>
Artefatos <code>sql/classificador_naive_bayes.sql</code> <code>sql/rodar_classificador.py</code>	

Este relatório explica a arquitetura do código para apoiar a defesa oral — não repete o SQL linha a linha (isso está nos comentários do próprio arquivo).

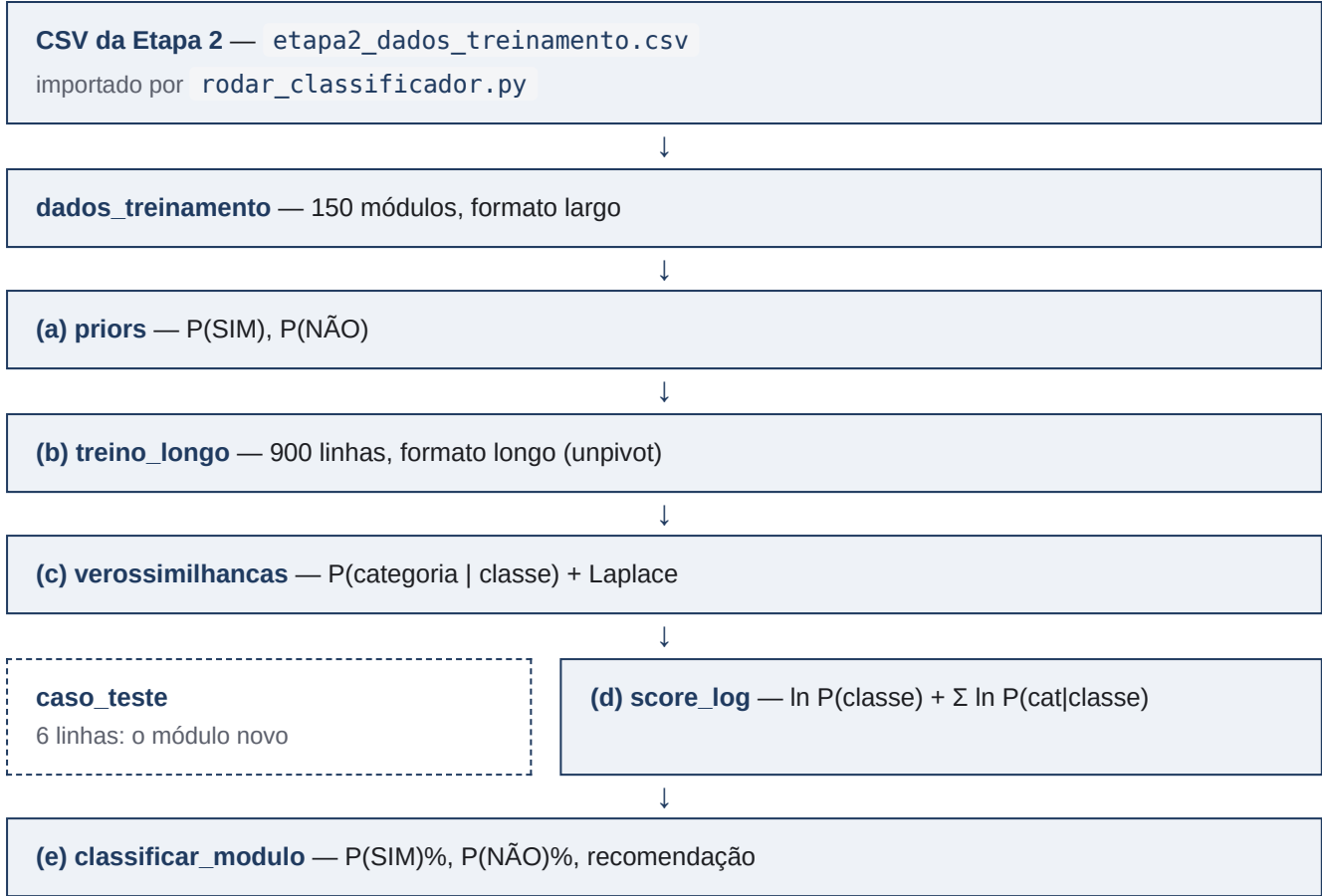
1 Por que SQLite

- **Roda sem servidor.** Um único arquivo (`classificador.db`); sem processo a subir, porta a abrir ou usuário/senha — só `import sqlite3`.
- **Fácil de testar no ambiente.** O script de teste cria o banco do zero, importa o CSV, roda o classificador e confere o resultado em segundos, sem dependências externas.
- **SQL padrão suficiente.** Usa apenas `CREATE VIEW`, `JOIN`, `GROUP BY`, CTEs (`WITH`), `UNION ALL`, `CASE` e as funções `LN` / `EXP` — tudo disponível no SQLite 3.45. Portar para PostgreSQL exigiria mudanças mínimas.

Custo assumido

SQLite tem tipagem fraca e não tem `PIVOT` nativo — o "unpivot" é feito à mão com `UNION ALL` (Seção 3.2), o que na verdade deixa a lógica mais explícita.

2 Fluxo de dados (do CSV ao veredito)



3 As views, uma a uma

3.1 priors — probabilidade a priori P(classe)

Conta quantos módulos de treino são SIM e quantos são NÃO e divide pelo total. É o palpite inicial, antes de olhar qualquer feature.

Resultado com a massa da Etapa 2: $P(\text{NÃO}) = 98/150 = 0,653$ e $P(\text{SIM}) = 52/150 = 0,347$.

3.2 treino_longo — "unpivot" das 6 categorias

`dados_treinamento` tem uma coluna de categoria por feature (`complexidade_cat` , `loc_cat` , ...). Para calcular a verossimilhança de todas as features com **uma** consulta em vez de seis quase iguais, empilhamos os seis recortes com `UNION ALL` , chegando ao formato longo:

(`modulo_id` , `defeito` , `feature` , `categoria`) — 150 módulos × 6 features = **900 linhas**.

O nome curto de cada feature (`complexidade` , `loc` , `n_autores` , `churn` , `n_imports` , `cobertura`) passa a ser a chave usada no resto do pipeline e também nos casos de teste.

3.3 verossimilhancas — P(categoria | classe) com suavização de Laplace

Para cada combinação (feature, categoria, classe):

$P(\text{feature} = \text{categoria} \mid \text{classe}) = (\text{contagem} + 1) / (\text{total_da_classe} + k)$, com $k = 3$ (baixo/médio/alto), igual para as 6 features.

Por que o "+1" e o "+k" (Laplace)

Sem eles, se uma categoria nunca aparece junto com uma classe no treino, a contagem é 0 e a probabilidade condicional é 0. Como o Naive Bayes combina as features multiplicando (aqui, somando os logs), **um único zero anula a classe inteira** — o modelo fica "certo demais" por causa de uma célula vazia que é só falta de dados. O Laplace adiciona meia folga a cada categoria: nenhuma probabilidade é exatamente 0 nem 1, e o efeito sobre categorias bem povoadas é desprezível. Bônus: as três probabilidades de cada (feature, classe) passam a somar exatamente 1.

A view gera primeiro **todas** as $6 \times 3 \times 2 = 36$ combinações possíveis (CROSS JOIN de features, categorias e classes) e só então faz `LEFT JOIN` com as contagens observadas — combinações ausentes no treino entram com contagem 0 e recebem o Laplace.

Exemplo concreto: `complexidade = alto` na classe SIM aparece 28 vezes; $P = (28 + 1) / (52 + 3) = 29/55 \approx 0,527$.

3.4 score_log — somar logaritmos em vez de multiplicar

Para cada classe, o Naive Bayes calcula $\text{score}(\text{classe}) = P(\text{classe}) \times P(\text{cat}_1 \mid \text{classe}) \times \dots \times P(\text{cat}_6 \mid \text{classe})$.

Por que log

Cada fator é menor que 1 (aqui entre ~0,02 e ~0,7). Multiplicar 7 números pequenos dá um valor minúsculo; com muitas features isso chega a **underflow** — o `float` vira 0 e a informação some. Tomando logaritmo, o produto vira **soma**:

$$\text{log score(classe)} = \ln P(\text{classe}) + \sum_i \ln P(\text{cat}_i \mid \text{classe})$$

Somar sete números da ordem de -1 a -4 é numericamente seguro. Como `ln` é crescente, a classe de maior log-score é a de maior probabilidade.

A view junta as 6 linhas do caso (`caso_teste`) com as verossimilhanças das duas classes e agrupa por (caso, classe), produzindo 2 linhas por caso. A coluna `n_features` deve dar 6 — se vier menor, alguma categoria do caso não casou.

3.5 `classificar_modulo` — normalização e recomendação

Converte os dois log-scores de volta para probabilidade entre 0 e 100%:

$$P(\text{SIM}) = \exp(\text{log_sim}) / (\exp(\text{log_sim}) + \exp(\text{log_nao}))$$

O denominador comum $P(\text{caso})$ desaparece nessa divisão; reexponenciar e dividir pela soma reintroduz a normalização. Antes do `exp()`, subtraímos `log_max = max(log_sim, log_nao)` dos dois expoentes — isso não muda o resultado (o fator sai igual no numerador e denominador), só evita passar argumentos muito negativos ao `exp()`. É a forma numericamente correta da conta.

Condição	Recomendação
<code>P(SIM) > 50%</code>	ALTO RISCO — revisão de código e testes adicionais
caso contrário	BAIXO RISCO — pode seguir o fluxo normal

4 Funções LN e EXP no SQLite

O SQLite ≥ 3.35 traz `ln()` e `exp()` nativas quando compilado com `SQLITE_ENABLE_MATH_FUNCTIONS`. O `sqlite3` do Python usado aqui (SQLite 3.45.1) já as inclui — confirmado com `SELECT ln(1), exp(0)`.

Portabilidade

Por segurança, `rodar_classificador.py` testa essas funções ao abrir a conexão e, se falharem, registra equivalentes em Python via `connection.create_function("ln", 1, math.log)` e `("exp", 1, math.exp)`. O mesmo `.sql` roda então em qualquer build, sem alteração.

5 Teste de ponta a ponta

`sql/rodar_classificador.py` recria o banco, roda o SQL e classifica **dois casos de exemplo mínimos** — apenas um *smoke test* para provar que o pipeline funciona. Os 5+ casos formais e a análise

crítica são da Etapa 4.

Caso	Perfil (6 categorias)	P(SIM)	P(NÃO)	Recomendação
exemplo_alto_risco	complexidade alto, loc baixo, autores alto, churn alto, imports alto, cobertura baixo	93,31%	6,69%	ALTO
exemplo_baixo_risco	complexidade baixo, loc medio, autores baixo, churn baixo, imports baixo, cobertura alto	4,17%	95,83%	BAIXO

Verificações automáticas do script

(falha ⇒ código de saída 1)

- As duas probabilidades somam **100,00%** nos dois casos.
- **6/6 features** casaram com a tabela de verossimilhanças.
- O veredito bate com a intuição do domínio: arquivo pequeno, denso, muito mexido e mal coberto → alto risco; arquivo médio, simples, estável e bem coberto → baixo risco.

Log-scores obtidos (log longe de qualquer underflow): `exemplo_alto_risco` → ln-score SIM -8,98 vs. NÃO -11,62 | `exemplo_baixo_risco` → ln-score SIM -9,45 vs. NÃO -6,32.

6 Como executar

```
python3 sql/rodar_classificador.py
```

O script é idempotente: recria `dados_treinamento` a partir do CSV e derruba/cria todas as views (`DROP ... IF EXISTS`), então pode ser rodado quantas vezes for preciso. Para inspecionar o modelo manualmente: `sqlite3 dados/classificador.db "SELECT * FROM verossimilhanças;"` .

Nota sobre o uso de IA

O SQL e o runner foram escritos em diálogo com uma IA generativa; cada view foi revisada e testada (contagens conferidas à mão, soma das verossimilhanças = 1, probabilidades finais somando 100%). Todo o código é comentado bloco a bloco e é defensável oralmente.